

Universal Artificial Intelligence

Chapter 7

Chapter 7 — Universal Artificial Intelligence

An Introduction to Universal Artificial Intelligence

Hutter, Quarel, Catt (2024)

“A sign of intelligence is an awareness of one's own ignorance.” — Niccolò Machiavelli

Outline

- 1 7.1 Acting Optimally in Known Environments
- 2 7.2 Bayesian Mixture of Environments
- 3 7.3 Acting Optimally in Unknown Environments
- 4 7.4 Universal Optimal Agent AIXI
- 5 7.5 Exercises
- 6 7.6 History and References

Recap: Agents, Environments, and Value (Chapter 6) I

This chapter builds directly on the agent framework of Chapter 6, so we first recall the cast of characters. Everything here is about an **agent** that repeatedly picks an **action** and receives back a **percept** from the world, forever.

- **History** $h_{<t} = a_1 e_1 a_2 e_2 \dots a_{t-1} e_{t-1}$: the complete record of everything the agent has done and observed strictly before time t .
- **Percept** $e_t = (o_t, r_t)$: what the world hands back at time t — an observation o_t together with a numerical reward $r_t \in [0, 1]$.
- **Environment** ν (nu): a probability distribution $\nu(e_{1:t} \| a_{1:t})$ over percept sequences given the agent's action sequence — i.e. the (possibly stochastic) rule the world uses to respond to actions. The double bar $\|$ marks this as a *chronological semimeasure*: probabilities that only ever depend on the past, never the future.
- **True environment** μ (mu): whichever single ν is actually generating the agent's experience. In this chapter, whether μ is known or unknown to the agent is exactly what distinguishes the different agents we study.

Recap: Agents, Environments, and Value (Chapter 6) II

- **Policy** π (pi): the agent's strategy, a function from histories to a (possibly randomized) choice of next action.
- **Value function** $V_{\nu}^{\pi}(h_{<t})$: the total expected future discounted reward if the agent follows policy π inside environment ν , starting from history $h_{<t}$.
- **Discount function** γ_k (gamma) and normalizer $\Gamma_t := \sum_{k=t}^{\infty} \gamma_k$: γ_k is the weight placed on the reward received at future time k ; discounting keeps the infinite sum of future rewards finite and expresses that sooner reward can matter more than later reward.
- **Q-value** $Q_{\nu}^{\pi}(h_{<t}, a_t)$: the value of taking action a_t right now and then following π from then on.
- **Optimal value** $V_{\nu}^*(h_{<t}) := \max_{\pi} V_{\nu}^{\pi}(h_{<t})$ and **optimal Q-value** $Q_{\nu}^*(h_{<t}, a_t)$: the best possible value/Q-value achievable by *any* policy in environment ν .

Recap: Agents, Environments, and Value (Chapter 6) III

Chapter 6 defined V and Q generally, for *any* choice of ν and π . This chapter is about specific, concrete choices of policy that try to actually maximize value, organized by how much the agent is told about the environment it is in:

- 1 **Section 7.1:** the environment μ is fully known $\Rightarrow$ agent **AI μ** .
- 2 **Section 7.2–7.3:** only a class $\mathcal{M}$ of candidate environments is known, combined into a single Bayes mixture ξ (xi) $\Rightarrow$ agent **AI ξ** .
- 3 **Section 7.4:** $\mathcal{M}$ is taken to be *every* computable environment, weighted by simplicity $\Rightarrow$ the universal agent **AIXI**.

Two Problems: Modelling and Planning

When an agent interacts with an environment to achieve its goals — usually phrased as maximizing its value function — there are two *distinct* sub-problems:

- 1 **Modelling**: figuring out what the environment actually is.
- 2 **Planning**: once the environment is known, figuring out the best way to act in it.

If the agent is simply handed the true environment μ (mu), the modelling problem is “solved” by assumption¹, leaving only the planning problem: choosing actions inside the known model μ to maximize the value function. Replacing ν by the true environment μ in the general definitions of Chapter 6, and letting the lookahead horizon $m \rightarrow \infty$, gives our first concrete agent: the fully **informed optimal agent**, called **AI** $_{\mu}$.

¹It could be that even when μ is given exactly, actually *computing* with it is intractable — but we set that concern aside in this section.

Definition 7.1.1: Policy of AI_μ

Definition 7.1.1 (Policy of AI_μ)

Given the true environment μ and some discount function $\gamma(\cdot)$, informed agent AI_μ selects an optimal policy

$$\pi_\mu^* \in \arg \max_{\pi} V_\mu^\pi \equiv \arg \max_{\pi} \lim_{m \rightarrow \infty} V_{\mu, \gamma}^{\pi, m}(\epsilon)$$

Plain English. π_μ^* (pi-mu-star) is *any* policy that attains the maximum possible μ -value: no other policy can do strictly better in environment μ . The right-hand side spells this out as a limit: $V_{\mu, \gamma}^{\pi, m}(\epsilon)$ is the value obtainable by planning only m steps ahead (starting from the empty history ϵ), and we let the lookahead horizon m grow to infinity so that no future reward, however far away, is ever left out of the calculation. AI_μ (“AI-mu”) is thus the best conceivable agent *given* perfect knowledge of the world it lives in — it is the yardstick every other agent in this chapter will be measured against.

Proposition 7.1.2: Action of AI_μ (the Expectimax Tree) I

Proposition 7.1.2 (Action of AI_μ)

The action a_t of AI_μ at time t given history $h_{<t}$ can be computed via

$$\begin{aligned} a_t &= a_t^{AI_\mu} := \arg \max_{a_t} Q_\mu^*(h_{<t}, a_t) \\ &= \arg \max_{a_t} \lim_{m \rightarrow \infty} \sum_{e_t} \max_{a_{t+1}} \sum_{e_{t+1}} \dots \max_{a_m} \sum_{e_m} \prod_{k=t}^m \mu(e_k | h_{<k} a_k) \sum_{k=t}^m \gamma_k r_k \end{aligned}$$

Plain English, term by term.

- $\arg \max_{a_t}(\cdot)$: the agent picks the action a_t right now that leads to the best outcome.
- $\sum_{e_t}(\cdot)$: the environment then responds with some percept e_t , and since the agent does not control e_t , we average (sum) over all its possible values, weighted by how likely μ says each one is.
- $\max_{a_{t+1}} \sum_{e_{t+1}} \dots$: this max/sum pattern repeats, one time step at a time, out to the horizon m — the agent alternately *chooses* (max) and the world *responds* (weighted sum).

Proposition 7.1.2: Action of AI_μ (the Expectimax Tree) II

- $\prod_{k=t}^m \mu(e_k | h_{<k} a_k)$: the probability, under the true environment μ , of the whole specific sequence of percepts along one particular branch of this decision tree.
- $\sum_{k=t}^m \gamma_k r_k$: the total discounted reward collected along that branch.

This alternating max/sum tree is called an **expectimax** tree — exactly like a minimax game tree from game theory, except the adversary (“chance”) is replaced by nature’s own probability distribution μ .

Computing AI_μ in Practice

Proof. This follows immediately from Lemma 6.6.3 and Definition 6.7.1 of Chapter 6, simply instantiated at $\nu = \mu$. ■

To directly compute $a_t^{AI_\mu}$ requires evaluating the expectimax tree truncated to an effective horizon $m = H_t(\varepsilon) < \infty$ (see Sections 6.7 and 12.1 for how large ε (epsilon) must be to guarantee near-optimal behaviour). In practice, this is computationally expensive — the tree has $|\mathcal{A}|^m \cdot |\mathcal{E}|^m$ leaves, roughly — so a variety of approximation techniques are used:

- **Pruning** or unrolling the expectimax calculation for only a short moving horizon.
- Using a hand-designed heuristic estimate $\hat{Q}$ of the true Q -value in place of computing it exactly [RN20].
- Learning $\hat{Q}$ from experience with a **neural network**, as in deep reinforcement learning systems.
- Approximating the expectimax with **Monte Carlo Tree Search (MCTS)**: several cheap *rollouts* (playing the “game” out to the end with a cheap proxy policy) are performed, and the resulting discounted sums of rewards are averaged to approximate the true expectation. This method is explored further in Section 12.2.2 of the book.

Python: Expectimax Planning for AI_μ

Proposition 7.1.2 made concrete on a tiny *known* 2-state environment: hot right after a reward of 1, cold right after a reward of 0; μ gives $P(\text{reward} = 1 \mid \text{state}, a)$ for each action $a \in \{0, 1\}$ — exactly what AI_μ was built for.

```
# Known environment mu: P(reward=1 | state, action)
mu = {('hot', 0): 0.5, ('hot', 1): 0.8,
      ('cold', 0): 0.5, ('cold', 1): 0.3}
next_state = lambda r: 'hot' if r == 1 else 'cold'

def expectimax(state, steps_left):
    # Prop. 7.1.2:
    # max (agent) / sum (world)
    if steps_left == 0:
        return 0.0, None
    best_a, best_v = None, -1.0
    for a in (0, 1):
        p1 = mu[(state, a)]
        # mu(e_t=1/h, a)
        v = sum(p*(r + expectimax(next_state(r), steps_left-1)[0])
                for r, p in ((1, p1), (0, 1-p1)))
        # sum over e_t
        if v > best_v: best_v, best_a = v, a
        # max over a_t
    return best_v, best_a

for m in (1, 2, 3):
    v, a = expectimax('hot', m)
    print(f"horizon m={m}: V*_mu(hot)={v:.4f}, a_1={a}")
```

Output: m=1: V*=0.8000, a_1=1, m=2: V*=1.5400, a_1=1, m=3: V*=2.2620, a_1=1. Even though action 1 only wins 80% of the time in hot, it also *keeps* the agent there more often than action 0, so it stays optimal at every horizon: expectimax correctly reasons several steps ahead, not just about immediate reward.

The Bayesian Solution to an Unknown Environment

In practice, the environment an agent interacts with is *not* fully known. The classical reinforcement-learning fix is to make some (strong) assumption about the environment and then run some heuristic algorithm to learn it from the agent-environment interaction.

The **Bayesian** solution instead: consider a whole *class* $\mathcal{M}$ of candidate environments, large enough that we are confident it contains the true environment μ ; devise a prior weight w_ν for every $\nu \in \mathcal{M}$; and use the known Bayes mixture ξ (x_i) in place of the unknown μ . This mirrors exactly the sequence-prediction story of Chapter 3, now lifted to an agent that can also *act*.

Definition 7.2.1: Chronological Bayes Mixture ξ

Definition 7.2.1 (Chronological Bayes mixture ξ)

Let $\mathcal{M}$ be a countable class of chronological semimeasures, and $w : \mathcal{M} \rightarrow \mathbb{R}$ be a prior satisfying $\sum_{\nu \in \mathcal{M}} w_\nu \leq 1$ and $w_\nu > 0$ for all $\nu \in \mathcal{M}$. We define the *chronological Bayes mixture* ξ over $\mathcal{M}$ given prior $w_{(\cdot)}$ as

$$\xi(e_{1:t} \| a_{1:t}) := \sum_{\nu \in \mathcal{M}} w_\nu \nu(e_{1:t} \| a_{1:t})$$

Plain English. ξ is simply a weighted average of all the candidate environments in $\mathcal{M}$, where each $\nu \in \mathcal{M}$ contributes in proportion to its prior weight w_ν — how plausible we thought ν was *before* seeing any data. Requiring $\sum_\nu w_\nu \leq 1$ keeps ξ itself a valid (chronological) semimeasure.

We will mainly restrict attention to countable policy spaces (e.g. *approximable* or *limit-computable* policies, Definition 2.6.13) rather than the uncountably many policies that exist in principle, since we are ultimately trying to build a physically realizable agent.

Definition 7.2.2: Bayesian Mixture as a Posterior

Definition 7.2.2 (Bayesian mixture ξ over environments)

Let $\mathcal{M}$ be a class of environments with prior $\{w_\nu\}_{\nu \in \mathcal{M}}$. We define $\xi(e_t|h_{<t}a_t)$ as the predictive probability the Bayesian mixture environment ξ assigns to percept e_t conditioned on history $h_{<t}$ and action a_t , recursively:

$$\begin{aligned}\xi(e_t|h_{<t}a_t) &:= \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \nu(e_t|h_{<t}a_t) \\ &= \frac{\xi(e_{1:t}||a_{1:t})}{\xi(e_{<t}||a_{<t})} = \frac{\sum_{\nu \in \mathcal{M}} w_\nu \nu(e_{1:t}||a_{1:t})}{\sum_{\nu \in \mathcal{M}} w_\nu \nu(e_{<t}||a_{<t})}, \quad \text{where} \\ w(\nu|h_{1:t}) &:= w(\nu|h_{<t}) \frac{\nu(e_t|h_{<t}a_t)}{\xi(e_t|h_{<t}a_t)} = w_\nu \frac{\nu(e_{1:t}||a_{1:t})}{\xi(e_{1:t}||a_{1:t})} \quad \text{and} \quad w(\nu|\epsilon) := w_\nu\end{aligned}$$

Plain English. $w(\nu|h_{<t})$ is the agent's *updated belief* that ν is the true environment, after seeing history $h_{<t}$; it starts at the prior w_ν and is refined by Bayes' rule every time a new percept arrives. This is exactly Theorem 3.1.7 from Chapter 3, but now every probability is additionally conditioned on the agent's own past actions $a_{<t}$ (and current action a_t), since here the “agent” actually gets to act rather than passively observe.

Example 7.2.3: A Three-Coin Mixture Environment I

Example 7.2.3 (Mixture environment)

Consider a coin-flip prediction problem with $\mathcal{A} = \mathcal{O} = \{H, T\}$, reward set $\{0, 1\}$, and class $\mathcal{M} = \{\nu_{HH}, \nu_{HT}, \nu_{TT}\}$ representing an environment that flips a two-headed coin, a fair coin, and a two-tailed coin, respectively. Each environment gives reward 1 to the agent if it correctly predicts the coin's outcome (i.e. if $a_t = o_t$). We choose the uniform prior $w_\nu = 1/3$.

Initially the mixture predicts the likelihood of Heads is $1/2$:

$$\xi(e_1=(H, 1)|h_{<1}=\epsilon, a_1=H) = \sum_{\nu \in \mathcal{M}} w_\nu \nu(e_1=(H, 1)|a_1=H) = \frac{1}{3} \cdot 1 + \frac{1}{3} \cdot \frac{1}{2} + \frac{1}{3} \cdot 0 = \frac{1}{2}$$

In fact, $\xi = \nu_{HT}$ exactly when the mixture is given an empty history — the fair-coin environment and the mixture agree, since a $1/2$ - $1/2$ mix of “always Heads” and “always Tails” looks just like a fair coin on the very first flip.

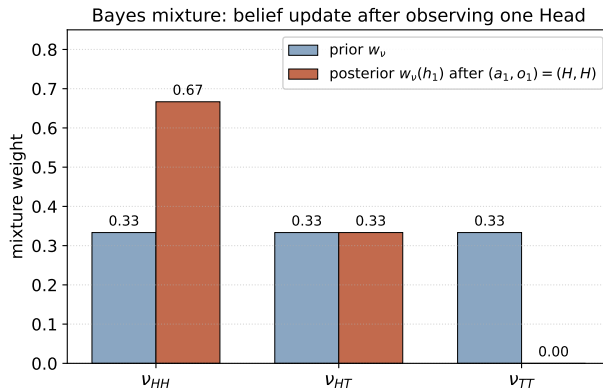
Example 7.2.3: A Three-Coin Mixture Environment II

The agent bets, and wins. Suppose the agent takes action $a_1 = H$ and guesses correctly, so the environment returns $(o_1, r_1) = (H, 1)$, giving history $h_1 = (a_1, e_1) = (H, (H, 1))$. Applying the recursive update rule of Definition 7.2.2:

$$w_{\nu_{HH}}(h_1) = \frac{1}{3} / \frac{1}{2} \cdot 1 = \frac{2}{3}, \quad w_{\nu_{HT}}(h_1) = \frac{1}{3} / \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{3}, \quad w_{\nu_{TT}}(h_1) = \frac{1}{3} / \frac{1}{2} \cdot 0 = 0$$

So the agent now believes with confidence $2/3$ that $\mu = \nu_{HH}$, with confidence $1/3$ that $\mu = \nu_{HT}$, and considers $\mu = \nu_{TT}$ *impossible* — which makes sense, since a two-tailed coin could never have produced the observed Head.

Example 7.2.3: A Three-Coin Mixture Environment III



Example 7.2.3: A Three-Coin Mixture Environment IV

Predicting further ahead. Using the updated weights, the dynamics of $\xi(e_2|h_1 a_2)$ become

$$\xi(e_2|h_1 a_2) = \sum_{\nu \in \mathcal{M}} w(\nu|h_1) \nu(e_2|h_1 a_2) = \frac{2}{3} \nu_{HH}(e_2|h_1 a_2) + \frac{1}{3} \nu_{HT}(e_2|h_1 a_2)$$

and the probability that the next observation is Heads, for either choice of action a_2 , is

$$\xi(o_2=H|h_1 H) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot \frac{1}{2} = \frac{5}{6}, \quad \xi(o_2=H|h_1 T) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot \frac{1}{2} = \frac{5}{6}$$

In both cases ξ predicts the next observation will be Heads with probability $5/6$, regardless of the action taken — consistent with this being a *passive* environment, where the agent's action only affects reward, not the coin. Note that $\xi \notin \mathcal{M}$: the mixture itself is not one of the three original environments, which is expected given how limited this toy model class is. ♦

Python: Reproducing the Coin-Mixture Example

```
from fractions import Fraction as F

# nu[k] = P(outcome = H), for k in {HH, HT, TT}
nu = {'HH': F(1,1), 'HT': F(1,2), 'TT': F(0,1)}
w = {k: F(1,3) for k in nu}          # uniform prior

# Agent bets a1=H, environment returns o1=H
like = {k: nu[k] for k in nu}
xi1 = sum(w[k]*like[k] for k in w)   # xi(o1=H|a1=H)
print("xi(o1=H|a1=H) =", xi1)        # -> 1/2

w_post = {k: w[k]*like[k]/xi1 for k in w}
for k in w_post:
    print(f"w_{k}(h1) =", w_post[k])
    # -> w_HH(h1)=2/3, w_HT(h1)=1/3, w_TT(h1)=0

xi2 = sum(w_post[k]*nu[k] for k in w_post)
print("xi(o2=H|h1) =", xi2)          # -> 5/6
```

Running this reproduces the book's numbers *exactly* (using exact fractions rather than floating point): a prior of $1/3$ on each of the three coins collapses, after a single observed Head, to posterior weights $2/3, 1/3, 0$ — and the mixture's prediction of the second flip being Heads rises to $5/6$.

Lemma 7.2.4: Linearity of the Posterior Mixture

Lemma 7.2.4 (Linearity of posterior ξ^π mixture)

For events $A \subseteq (\mathcal{A} \times \mathcal{E})^\infty$ and functions $f : (\mathcal{A} \times \mathcal{E})^\infty \rightarrow \mathbb{R}$,

$$P_\xi^\pi[A|h_{<t}] = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) P_\nu^\pi[A|h_{<t}] \quad \text{and} \quad \mathbf{E}_\xi^\pi[f|h_{<t}] = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \mathbf{E}_\nu^\pi[f|h_{<t}]$$

Linearity with the same weights $w(\nu|h_{<t})$ remains true if we further condition on a_t . A particular instantiation is

$$\xi^\pi(h_{1:m}) = \prod_{t=1}^m \pi(a_t|h_{<t}) \nu(e_t|h_{<t}a_t) = \sum_{\nu \in \mathcal{M}} w_\nu \nu^\pi(h_{1:m}) \geq w_\mu \mu^\pi(h_{1:m}) \quad \forall \mu \in \mathcal{M}$$

Plain English. Any probability or expectation computed under the mixture ξ , given a fixed policy π , is simply the corresponding probability/expectation under each individual ν , averaged using the *posterior* weights $w(\nu|h_{<t})$ — exactly as one would hope from an average. The final inequality is the agent-case analogue of “ ξ dominates every $w_\mu \mu$ ” from Chapter 3: the mixture’s probability of any full history is never smaller than any single environment’s probability of it, scaled by that environment’s prior weight.

Theorem 7.2.5: Linearity and Convexity of the Value Function

Theorem 7.2.5 (Linearity/convexity of V)

$V_\nu^\pi(h_{<t})$ is linear in ν and $V_\xi^*(h_{<t})$ is convex in ν :

$$V_\xi^\pi(h_{<t}) = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) V_\nu^\pi(h_{<t}) \quad \text{and} \quad V_\xi^*(h_{<t}) \leq \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) V_\nu^*(h_{<t})$$

The same holds for the Q-values $Q_\nu^\pi(h_{<t}a_t)$ with the same weights $w(\nu|h_{<t})$.

Plain English. The *fixed-policy* value V^π behaves exactly like a weighted average (equality), because following one policy π throughout is compatible with every ν simultaneously. But the *optimal* value V^* only satisfies an inequality ($\leq$): on the left, one single policy π_μ^* must be tailored to μ alone; on the right, a single fixed policy must perform decently in every $\nu \in \mathcal{M}$ at once inside the mixture, which is a strictly harder ask, hence the mixture value can only be lower.

Theorem 7.2.6: Merging of Opinions

As $t \rightarrow \infty$ we would hope that $\xi \rightarrow \mu$ in a strong predictive sense, just as in Chapter 3's Corollary 3.3.2 (one-step predictive convergence). Two complications arise for an *acting* agent: we now need *multi-step* and *off-policy* convergence, and discounted values require looking infinitely far ahead. It turns out this still holds, but without any guaranteed convergence *rate*:

Theorem 7.2.6 (Merging of opinions [BD62])

$$\sup_A |P_\xi^\pi[A|h_{<t}] - P_\mu^\pi[A|h_{<t}]| \xrightarrow{t \rightarrow \infty} 0 \quad \mu^\pi\text{-a.s.}$$

$$\sup_A |P_\xi^\pi[A|h_{<t}a_t] - P_\mu^\pi[A|h_{<t}a_t]| \xrightarrow{t \rightarrow \infty} 0 \quad \mu^\pi\text{-a.s.}$$

where $\sup_A$ ranges over all measurable events $A \subseteq (\mathcal{A} \times \mathcal{E})^\infty$, and $P_\nu^\pi[A|h_{<t}] = \mathbf{E}_\nu^\pi[\mathbb{I}_A|h_{<t}]$.

Plain English. No matter which *measurable* question A you ask about the entire infinite future, the mixture ξ 's answer and the true environment μ 's answer eventually agree — almost surely, when histories are sampled by running policy π in the true environment μ (written μ^π). Because this holds for every such A simultaneously, it immediately implies both single-step convergence ($\xi(e_t|h_{<t}a_t) \rightarrow \mu(e_t|h_{<t}a_t)$) and m -step-ahead convergence for any fixed m . The figure a few slides ahead (after Theorem 7.3.6) plots this prediction gap shrinking towards 0 in a small simulated example.

From Predictive Convergence to Value Convergence

Since $\xi \rightarrow \mu$ as $t \rightarrow \infty$ (Theorem 7.2.6), it is plausible to conjecture that the *value function* V_ξ also converges to V_μ , regardless of which policy is used. Even if the agent always guessed the wrong action, it would still gather evidence about which environment it is really in, so it would eventually learn the truth anyway — just without necessarily earning much reward while doing so.

On-policy convergence. If the agent actually follows a fixed policy π while the true environment is μ , then as $t \rightarrow \infty$ the computed value $V_\xi^\pi(h_{<t})$ converges to the true value $V_\mu^\pi(h_{<t})$ — with probability 1 under μ^π (i.e. this failing has probability exactly zero).

Theorem 7.3.1: On-Policy Value Convergence of Bayes

Theorem 7.3.1 (On-policy value convergence of Bayes)

For any environment $\mu \in \mathcal{M}$ and any policy π ,

$$V_{\xi}^{\pi}(h_{<t}) - V_{\mu}^{\pi}(h_{<t}) \longrightarrow 0 \quad \text{for } t \rightarrow \infty \quad \mu^{\pi}\text{-almost surely}$$

Proof sketch. Instantiate the general inequality

$$|\mathbf{E}_Q[f] - \mathbf{E}_P[f]| \leq \sup |f| \cdot \sup_A |Q[A] - P[A]|$$

with $f = \sum_{k=t}^{\infty} \gamma_k r_k / \Gamma_t$ (the normalized future discounted reward), $P = P_{\mu}^{\pi}(\cdot|h_{<t})$, and $Q = P_{\xi}^{\pi}(\cdot|h_{<t})$. This gives

$$|V_{\xi}^{\pi}(h_{<t}) - V_{\mu}^{\pi}(h_{<t})| \leq \sup |f| \cdot \sup_A |P_{\xi}^{\pi}[A|h_{<t}] - P_{\mu}^{\pi}[A|h_{<t}]| \xrightarrow{t \rightarrow \infty} 0 \quad \mu^{\pi}\text{-a.s.}$$

The convergence follows since $\sup |f| \leq \sup |\mathcal{R}| \leq 1$ (rewards are bounded) and by Theorem 7.2.6 (merging of opinions). Convergence rates similar to, but weaker than, those of Chapter 3, Section 3.5, can also be obtained. ■

Definition 7.3.2: The Bayes-Optimal Agent AI_ξ

We are ultimately interested in measuring an agent's performance against V_μ^* , the optimal value function when μ is known and the best policy π_μ^* is used. Since μ is actually unknown, the best policy the agent *can* choose is the one that maximizes its own best estimate V_ξ^π of the environment, since ξ represents the agent's best guess at μ given both the prior w_ν and everything observed so far.

Definition 7.3.2 (Bayes-optimal agent AI_ξ)

$$\pi_\xi^* := \arg \max_{\pi} V_\xi^\pi$$

Plain English. AI_ξ (“AI-xi”) behaves exactly like AI_μ from Section 7.1, except it plans against its own current *belief* ξ about the world instead of the (unknown) true environment μ . Representation Proposition 7.1.2 carries over unchanged with μ replaced by ξ . Theorem 7.3.1 is reassuring, but it does *not* yet tell us anything about how the value of *this specific* policy π_ξ^* , evaluated in the true environment μ , compares to the truly optimal value V_μ^* .

Definition 7.3.3: Self-Optimizing Policies

What we really want to know is whether $V_{\mu}^{\pi_{\xi}^*}(h_{<t})$ converges to $V_{\mu}^*(h_{<t})$ — i.e. whether the true value achieved by the Bayes-optimal policy π_{ξ}^* (the policy the agent actually has to use, in the face of an unknown μ) converges to the value of the best policy π_{μ}^* it *could* have used, had it known μ all along. In control theory this property is called **self-optimizing**:

Definition 7.3.3 (Self-optimizing policies)

A policy $\tilde{\pi}$ is called *self-optimizing* for a class $\mathcal{M}$ and historic policy π if

$$\forall \nu \in \mathcal{M} : \quad V_{\nu}^*(h_{<t}) - V_{\nu}^{\tilde{\pi}}(h_{<t}) \xrightarrow{t \rightarrow \infty} 0 \quad \text{with } \nu^{\pi} \text{ probability 1}$$

Plain English. $\tilde{\pi}$ (pi-tilde) eventually performs just as well as the best possible policy, *in every single environment* $\nu \in \mathcal{M}$ that could be the truth — no matter which policy π was used to generate the historic data $\tilde{\pi}$ is learning from. This is a strong, uniform requirement: while the future percepts $e_{1:\infty}$ are sampled from the real ν , the *past* actions $a_{<t}$ can be utterly arbitrary.

Lemma 7.3.4: Convergence of Posterior Averages

Lemma 7.3.4 (Convergence of posterior averages [Lat23])

For any policy π , if $0 \leq \delta_{\nu,t}(h_{<t}) \rightarrow 0$ for $t \rightarrow \infty$ with ν^π -probability 1, for all $\nu \in \mathcal{M}$, then

$$\sum_{\nu \in \mathcal{M}} w_\nu \frac{\nu(e_{<t} \| a_{<t})}{\mu(e_{<t} \| a_{<t})} \delta_{\nu,t}(h_{<t}) \xrightarrow{t \rightarrow \infty} 0 \quad \text{with } \mu^\pi\text{-probability 1}$$

and equivalently $\sum_{\nu \in \mathcal{M}} w(\nu | h_{<t}) \delta_{\nu,t}(h_{<t}) \xrightarrow{t \rightarrow \infty} 0$ with μ^π -probability 1.

Plain English. $\delta_{\nu,t}$ (delta) is meant to represent a “gap” associated with environment ν at time t that shrinks to zero for every individual ν . This lemma says that a whole *weighted average* of such gaps, weighted either by the likelihood ratio ν/μ or (equivalently) by the posterior $w(\nu|h_{<t})$, also shrinks to zero — a countable weighted sum of vanishing terms still vanishes, even though there are infinitely many terms and the weights themselves are changing over time. The proof (omitted here) uses that $X_t := \nu(e_{<t} \| a_{<t}) / \mu(e_{<t} \| a_{<t})$ forms a μ -supermartingale, together with a reverse Fatou-lemma argument.

Theorem 7.3.6: Self-Optimizing — Statement and Proof I

Theorem 7.3.6 (Self-optimizing [Hut05b])

Let $\mathcal{M}$ be a countable class of environments and π be any historic policy. If there exists a self-optimizing policy $\tilde{\pi}$ for $\mathcal{M}$ in the sense of Definition 7.3.3, then the Bayes-optimal policy π_{ξ}^* is self-optimizing for $\mathcal{M}$.

Why this matters. This is one of the central results of the book. Obviously, if no self-optimizing $\tilde{\pi}$ exists at all, we cannot expect π_{ξ}^* to be one either. So the sufficient condition here is also *necessary* — as weak an assumption as possible. Its strength compared to the asymptotic-optimality results of Chapter 8 is that it is **off-policy**: π_{ξ}^* ends up self-optimizing even if the agent followed a completely different policy $\pi \neq \pi_{\xi}^*$ in the past.

Theorem 7.3.6: Self-Optimizing — Statement and Proof II

Proof. For each $\nu \in \mathcal{M}$ define $\delta_{\nu,t}(h_{<t}) := V_{\nu}^*(h_{<t}) - V_{\nu}^{\tilde{\pi}}(h_{<t})$. Since rewards are bounded by 1 and V_{ν}^* is a weighted average of rewards, $0 \leq V_{\nu}^{\tilde{\pi}}(h_{<t}) \leq V_{\nu}^*(h_{<t}) \leq r_{\max} = 1$, which implies $0 \leq \delta_{\nu,t} \leq 1$. Additionally,

$$\begin{aligned} 0 &\leq w(\mu|h_{<t}) [V_{\mu}^*(h_{<t}) - V_{\mu}^{\pi_{\xi}^*}(h_{<t})] \\ &\leq \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) [V_{\nu}^*(h_{<t}) - V_{\nu}^{\pi_{\xi}^*}(h_{<t})] \\ &\leq \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) [V_{\nu}^*(h_{<t}) - V_{\nu}^{\tilde{\pi}}(h_{<t})] = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \delta_{\nu,t}(h_{<t}) \end{aligned}$$

The first inequality holds because $V_{\mu}^* \geq V_{\mu}^{\pi_{\xi}^*}$ (the optimal value for μ is at least the value of any other policy). The second follows from adding only non-negative terms. The third follows because $\sum_{\nu} w(\nu|h_{<t}) V_{\nu}^{\pi_{\xi}^*}(h_{<t}) = V_{\xi}^{\pi_{\xi}^*}(h_{<t}) = V_{\xi}^*(h_{<t}) \geq V_{\xi}^{\tilde{\pi}}(h_{<t}) = \sum_{\nu} w(\nu|h_{<t}) V_{\nu}^{\tilde{\pi}}(h_{<t})$ (Theorem 7.2.5, plus optimality of π_{ξ}^* for ξ).

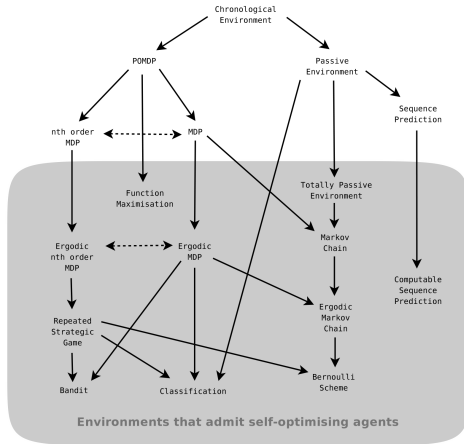
Theorem 7.3.6: Self-Optimizing — Statement and Proof III

Dividing the chain of inequalities above by $w(\mu|h_{<t})$:

$$0 \leq V_{\mu}^*(h_{<t}) - V_{\mu}^{\pi_{\xi}^*}(h_{<t}) \leq \frac{1}{w_{\mu}} \sum_{\nu \in \mathcal{M}} w_{\nu} \frac{\nu(e_{<t} \| a_{<t})}{\mu(e_{<t} \| a_{<t})} \delta_{\nu,t}(h_{<t})$$

which by Lemma 7.3.4 converges to 0 as $t \rightarrow \infty$ with μ^{π} -probability 1. Since this holds for every $\mu \in \mathcal{M}$, π_{ξ}^* is self-optimizing. ■

Figure 7.1: Which Environment Classes Admit Self-Optimizing Agents?



Reading the Taxonomy

Downward arrows in Figure 7.1 (previous slide) mean “the class below is a special case of the class above.” Dotted horizontal lines mean two classes are *reducible* to each other. The **grey region** contains exactly the environment classes that *do* admit self-optimizing agents — ergodic n th-order MDPs (Markov Decision Processes) are one large such class, hence π_ξ^* is self-optimizing for them by Theorem 7.3.6. Sequence prediction is Chapter 3's topic; Markov chains, Chapters 4–5; (ergodic) n th-order MDPs, Chapters 11, 12, 14.

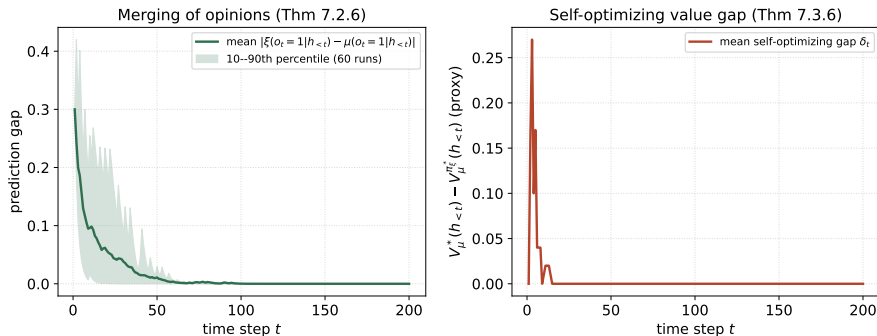
Self-Optimizing Fails Without Ergodicity: Traps

Unfortunately, *without* some form of **ergodicity** assumption — roughly, the requirement that the agent can always find its way back to explore any part of the environment again — self-optimizing fails, because the agent can fall into **traps**: parts of the state space from which it can never recover, so early mistakes become permanent. The book’s “Heaven-Hell” example (Example 8.1.3, covered in Chapter 8) illustrates this concretely: one wrong early action locks the agent forever into a low-reward region it can never escape.

Consequence. The class of *all* (semi)computable chronological environments — with no ergodicity restriction — does **not** admit self-optimizing policies. This is exactly the gap that motivates weaker, *on-average* notions of optimality, which Chapter 8 develops.

Convergence in Practice: Merging of Opinions and Self-Optimizing

The two theorems of this section — merging of opinions (Thm. 7.2.6) and self-optimizing (Thm. 7.3.6) — both describe quantities shrinking to 0 over time, simulated below on a small three-environment Bernoulli mixture (uniform prior) acting greedily in a fixed true environment μ .



Left: the prediction gap $|\xi(o_t=1|h_{<t}) - \mu(o_t=1|h_{<t})|$ (Thm. 7.2.6). **Right:** the self-optimizing value gap $\delta_t = V_\mu^*(h_{<t}) - V_\mu^{\pi_\xi^*}(h_{<t})$ (Thm. 7.3.6). Both shrink towards 0 once the mixture is confident which environment it faces.

From $AI\xi$ to $AIXI$: the Universal Mixture

So far we have considered a *general* environment class $\mathcal{M}$ and a *general* prior w_ν . We can now combine our earlier choice of class and prior (motivated by Occam's razor and Epicurus' principle of multiple explanations, Section 3.7) with the notion of a Bayesian agent, arriving at the **universal Bayesian agent $AIXI$** .

$AI\xi$ from the previous section is exactly like $AI\mu$, except it does not know the true environment, so we replaced μ with ξ , the Bayesian mixture over $\mathcal{M}$. We now define **$AIXI$** to be the universal Bayesian agent that uses the class of *all* lower-semicomputable chronological semimeasures $\mathcal{M} = \mathcal{M}_{sol}$, together with the **universal prior** $w_\nu = w_\nu^U := 2^{-K(\nu)}$ and mixture distribution $\xi = \xi_U$. Here $K(\nu)$ is the **Kolmogorov complexity** of ν — the length, in bits, of the shortest computer program that computes ν — so simpler (shorter-description) environments get exponentially more prior weight: Occam's razor, made precise.

Definition 7.4.1: The Universal Optimal Agent AIXI

Definition 7.4.1 (Universal optimal agent AIXI)

AIXI is the Bayes-optimal agent for the universal mixture $\xi_U(\cdot) := \sum_{\nu \in \mathcal{M}_{sol}} w_\nu^U \nu(\cdot)$, with universal prior $w_\nu^U := 2^{-K(\nu)}$:

$$\pi^{AIXI} := \pi_{\xi_U}^* = \arg \max_{\pi} V_{\xi_U}^{\pi}$$

Plain English. AIXI is precisely AI_{ξ} from Section 7.3, with the class $\mathcal{M}$ specialized to *every computable environment* and the prior specialized to the *universal prior* $2^{-K(\nu)}$. There is no further tuning knob left to choose: this is the most environment-independent Bayesian agent conceivable.

AIXI is regarded as the most “intelligent” environment-independent agent possible because it optimally learns and acts across an enormous range of environments, adapts to whatever situation it finds itself in, and combines all hypotheses according to Occam’s razor. As it interacts with its environment, it continuously updates its beliefs $w(\nu|h_{<t})$, gathering more information over time and refining its understanding, which lets it make progressively better-informed decisions.

Theorem 7.4.2: Expectimax Bayesian Form of AIXI

Theorem 7.4.2 (Expectimax Bayesian form of AIXI)

For finite lifetime m and no discounting, i.e. $\gamma_k = \mathbb{1}[k \leq m]$, given interaction history $h_{<t}$, the action of AIXI at time t has the explicit expectimax expression

$$a_t = \pi_{\xi_U}^*(h_{<t}) = \arg \max_{a_t} \sum_{e_t} \max_{a_{t+1}} \sum_{e_{t+1}} \dots \max_{a_m} \sum_{e_m} \left[\sum_{i=t}^m r_i \right] \sum_{\nu \in \mathcal{M}_{sol}} w_\nu^U(h_{<t}) \prod_{k=t}^m \nu(e_k | h_{<k} a_k)$$

where $w_\nu^U(h_{<t})$ is the posterior (Definition 7.2.2) for universal prior $w_\nu^U(\epsilon) := 2^{-K(\nu)}$.

Plain English. $\mathbb{1}[k \leq m]$ (the indicator function) is 1 for $k \leq m$ and 0 otherwise: rewards count in full up to the fixed horizon m and not at all beyond it — “no discounting” up to a cutoff. Everything else is the same expectimax alternation as Proposition 7.1.2, except the single known μ has been replaced by an explicit sum over the *entire* universal class $\mathcal{M}_{sol}$, each term weighted by the current posterior belief $w_\nu^U(h_{<t})$ in that environment.

Proof of Theorem 7.4.2

Proof. The explicit expression for the action $a_t = \pi_\nu^*(h_{<t})$ of AI_ν was given in Chapter 6 (Eq. 6.7.7). Replacing ν by ξ gives AI_ξ . The key step is rewriting the product of ξ -probabilities purely in terms of the individual ν 's:

$$\begin{aligned} \prod_{k=t}^m \xi(e_k | h_{<k} a_k) &= \frac{\xi(e_{1:m} \| a_{1:m})}{\xi(e_{<t} \| a_{<t})} = \frac{\sum_\nu w_\nu \nu(e_{1:m} \| a_{1:m})}{\xi(e_{<t} \| a_{<t})} \\ &= \sum_\nu w_\nu \frac{\nu(e_{<t} \| a_{<t})}{\xi(e_{<t} \| a_{<t})} \prod_{k=t}^m \nu(e_k | h_{<k} a_k) = \sum_\nu w(\nu | h_{<t}) \prod_{k=t}^m \nu(e_k | h_{<k} a_k) \end{aligned} \quad (7.4.3)$$

where we used Definitions 7.2.1 and 7.2.2 twice. AI_XI is simply the special case for $\mathcal{M}_{sol}$, w_ν^U , and ξ_U , which proves the theorem. ■

There is yet another — arguably the most elegant — formulation of AI_XI , based on a variant of the monotone Turing machine, which we define next.

Definition 7.4.4: Chronological Turing Machine

Definition 7.4.4 (Chronological Turing machine)

A (monotone) Turing machine T is called a *chronological Turing machine* if, when fed a program p , it transforms an input data stream $a_{1:m}$ into an output data stream $e_{1:m}$ such that for all $k \leq m$, it must print each “percept” e_k on the output tape prior to reading the next “action” a_{k+1} from the input tape. Formally $p', a_k, o_k, r_k, e_k \equiv o_k r_k$ are all understood to be encoded prefix-free, so that they can be uniquely decoded from $a_{1:m}$ and $e_{1:m}$. Notationally we define

$$T(p' a_{1:m}) \rightarrow e_{1:m} \text{ is true} \quad \text{iff} \quad T(p' a_{1:k}) = e_{1:k} \text{ for all } k \leq m$$

Plain English. This is just a careful bookkeeping device: it formalizes the “chronological” requirement (percepts printed before the next action is read) inside a program that can also read inputs *interactively*, so we can talk rigorously about a Turing machine that plays the role of an agent’s “model” of the environment while respecting the actual time ordering of the interaction.

Theorem 7.4.5: Expectimax Solomonoff Form of AIXI

Theorem 7.4.5 (Expectimax Solomonoff form of AIXI)

The action of undiscounted horizon- m AIXI at time t given interaction history $h_{<t}$ is:

$$a_t^* = \pi_M^*(h_{<t}) = \arg \max_{a_t} \sum_{e_t} \dots \max_{a_m} \sum_{e_m} [r_t + \dots + r_m] M(e_{1:m} \| a_{1:m})$$

where

$$M(e_{1:m} \| a_{1:m}) := \sum_{p: U(p' a_{1:m}) \rightarrow e_{1:m}} 2^{-\ell(p)}$$

is a chronological version (Definition 6.3.1) of Solomonoff's distribution (3.8.3), and U is a chronological Universal Turing machine (Definition 7.4.4).

Plain English. $\ell(p)$ is the length, in bits, of program p . M sums $2^{-\ell(p)}$ over every program p that, when combined with the agent's action tape $a_{1:m}$, causes the universal machine U to output exactly the percept sequence $e_{1:m}$: shorter programs contribute exponentially more weight, so this is a direct algorithmic (program-length-based) restatement of the same universal-prior idea, this time phrased over programs rather than over environments.

Proof of Theorem 7.4.5: Equivalence with the Bayesian Form

Proof. The inner sum of Theorem 7.4.2 can be written as

$$\sum_{\nu \in \mathcal{M}} w(\nu | h_{<t}) \prod_{k=t}^m \nu(e_k | h_{<k} a_k) = \frac{\xi(e_{1:m} \| a_{1:m})}{\xi(e_{<t} \| a_{<t})} \quad (7.4.6)$$

where $\xi(e_{1:m} \| a_{1:m}) := \sum_{\nu \in \mathcal{M}} w_{\nu} \nu(e_{1:m} \| a_{1:m})$ (7.4.7) and $\nu(e_{1:m} \| a_{1:m}) := \prod_{k=1}^m \nu(e_k | h_{<k} a_k)$ (7.4.8). Since the denominator $\xi(e_{<t} \| a_{<t})$ in (7.4.6) does not depend on anything after time $t-1$, it can be pulled entirely out of the expectimax expression and, since it does not affect the $\arg \max_{a_t}$, ignored altogether — leaving only the numerator $\xi(e_{1:m} \| a_{1:m})$ to consider.

For the universal choice $w_{\nu}^U := 2^{-K(\nu)}$, i.e. ξ_U , one can show (analogously to Theorem 3.8.8) that

$$M(e_{1:m} \| a_{1:m}) \stackrel{\times}{=} \xi_U(e_{1:m} \| a_{1:m}) \quad (7.4.9)$$

(the $\stackrel{\times}{=}$ symbol means “equal up to a universal multiplicative constant”). Hence, up to two irrelevant multiplicative factors, the l.h.s. of (7.4.6) equals $M(e_{1:m} \| a_{1:m})$, so the latter can replace the former in Theorem 7.4.2, proving Theorem 7.4.5. ■

Theorem 7.4.10: AIXI Cannot Act Poorly in Good Environments I

Theorem 7.4.10 (AIXI cannot act poorly in good environments)

If $\mu \in \mathcal{M}$ is deterministic and there exists an $\varepsilon > 0$ such that for some $h_{1:\infty}$ we have $V_\mu^*(h_{<t}) > \varepsilon$ for all t , then

$$V_\xi^*(h_{<t}) \not\rightarrow 0 \quad \text{for } t \rightarrow \infty$$

This is true in particular for the history $h_{1:\infty}$ deterministically generated from (deterministic) μ and (deterministic) π_ξ^* .

Plain English. If the world is secretly “good” — there is always at least ε worth of value on the table, achievable by a policy that knows μ exactly — then the mixture’s own optimal-value estimate V_ξ^* can never be fooled into thinking there is *nothing* worth doing (i.e. it never drifts down to 0). AIXI’s own optimism about its situation is never unjustified when the situation is genuinely good.

Proof. Using convexity of V^* (Theorem 7.2.5) and Definition 7.2.2:

$$V_\xi^*(h_{<t}) \geq \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) V_\nu^*(h_{<t}) \geq w_\mu \frac{\prod_{k=1}^{t-1} \mu(e_k|h_{<k}a_k)}{\prod_{k=1}^{t-1} \xi(e_k|h_{<k}a_k)} V_\mu^*(h_{<t}) > \varepsilon w_\mu$$

Theorem 7.4.10: AIXI Cannot Act Poorly in Good Environments II

since $\prod_{k=1}^{t-1} \mu(e_k | h_{<k} a_k) = 1$ because μ is deterministic, $\xi(e_k | h_{<k} a_k) \leq 1$, and $V_\mu^*(h_{<t}) > \varepsilon$ by assumption. ■

Python: Universal Prior and a Toy AIXI Decision

Occam's razor made numeric: a handful of toy candidate environments with hand-assigned Kolmogorov complexities $K(\nu)$, combined via $w_\nu^U = 2^{-K(\nu)}$ (renormalized here since our toy class is finite, unlike the real, countably infinite $\mathcal{M}_{sol}$).

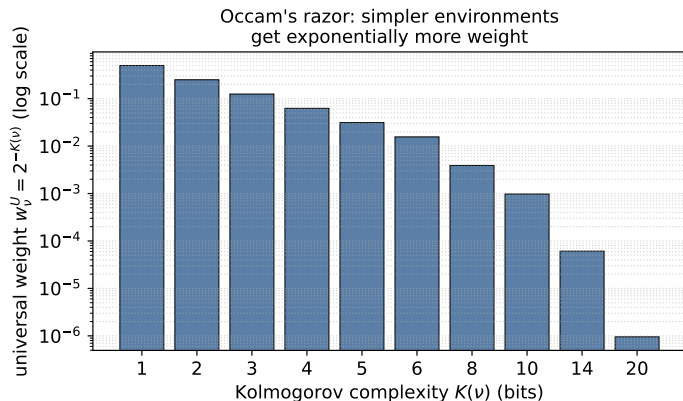
```
envs = { # K(nu) in bits; P(reward=1 | action A or B)
    'nu1': {'K': 2, 'A': 0.9, 'B': 0.1}, # simple: A wins
    'nu2': {'K': 3, 'A': 0.2, 'B': 0.9}, # simple: B wins
    'nu3': {'K': 5, 'A': 0.5, 'B': 0.5}, # complex "noise"
}
raw_w = {k: 2.0**(-envs[k]['K']) for k in envs}
Z = sum(raw_w.values())
w = {k: raw_w[k]/Z for k in envs} # w_nu^U, normalised

for k in envs:
    print(f"{k}: K={envs[k]['K']}, w^U={w[k]:.4f}")
for a in ('A', 'B'):
    xi_a = sum(w[k]*envs[k][a] for k in envs)
    print(f"xi_U(reward=1|a={a}) = {xi_a:.4f}")
```

Output: $w_{\nu_1}^U=0.6154$, $w_{\nu_2}^U=0.3077$, $w_{\nu_3}^U=0.0769$; $\xi_U(\text{reward}=1|A)=0.6538$, $\xi_U(\text{reward}=1|B)=0.3769$
 $\Rightarrow$ AIXI picks action A. Even though ν_2 argues strongly for action B, its higher complexity ($K=3$ vs. $K=2$) gives it less than half the weight of ν_1 , so the simpler hypothesis wins the vote.

Occam's Razor, Visualized

The same idea, zoomed out: as the description length (Kolmogorov complexity) $K(\nu)$ of a candidate environment grows, its universal prior weight $w_\nu^U = 2^{-K(\nu)}$ shrinks *exponentially*. Ten extra bits of complexity costs a factor of $2^{10} \approx 1000$ in prior weight — a steep, automatic penalty against overly complicated hypotheses, with no separately-tuned “complexity penalty” required.



Exercises I

1. [C15] Explicate the proof of Definition 7.2.2 by lifting the proof of Theorem 3.1.7.
2. [C15] Use Blackwell & Dubin's Theorem 3.9.5 to prove the agent version Theorem 7.2.6.
3. [C21i] Complete the proof of Theorem 7.3.1.
4. [C15i] This exercise extends Definition 3.9.1 and Example 3.9.2 to the agent case: random variables $X_t = X_t(h_{1:\infty})$ are said to form a supermartingale if $\mathbf{E}[X_t|h_{<t}] \leq X_{t-1}$. Show that $X_t := \nu(e_{1:t}||a_{1:t})/\mu(e_{1:t}||a_{1:t})$ in the proof of Lemma 7.3.4 is a μ^π -supermartingale.
5. [C30m] Show that X_t from the previous exercise converges to a finite constant w. μ .p.1.
6. [C30u] Show that for every enumerable chronological semimeasure ρ there exists a Turing machine T of length $\ell(T) = K(\rho) + O(1)$ that computes it, i.e. $\rho(h_{1:t}) = \sum_{q: T(q, a_{1:t}) \rightarrow e_{1:t}} 2^{-\ell(q)}$.
7. [C20u] Let $\delta(t) = \sum_{\nu \in \mathcal{M}} w_\nu \delta_\nu(t)$ and $\sum_{\nu \in \mathcal{M}} w_\nu \leq 1$. Show that the boundedness assumption $0 \leq \delta_\nu(t) \leq c$ is necessary for $\delta_\nu(t) \rightarrow 0$ as $t \rightarrow \infty$ to imply existence and/or convergence of $\delta(t)$. Show that $\delta_\nu(t) = O(f(t)) \forall \nu \in \mathcal{M}$ does not necessarily imply $\delta(t) = O(f(t))$ if $\mathcal{M}$ is infinite, even for bounded δ_ν .

Exercises II

8. [C30ui] Consider an environmental class $\mathcal{M}$ that admits self-optimizing policies. Study the effect of additionally believing in some $\rho \notin \mathcal{M}$ with small probability α (alpha): the new belief prior is $\xi' := (1 - \alpha)\xi + \alpha\rho$. Show that a belief α in ρ much smaller than the belief w_μ in the true environment $\mu \in \mathcal{M}$ only causes a small corruption of the self-optimizing property. More precisely, show
- $$\limsup_{t \rightarrow \infty} [V_\mu^*(h_{<t}) - V_\mu^{\pi_{\xi'}^*}(h_{<t})] \leq \frac{\alpha}{(1-\alpha)w_\mu}.$$
9. [C40o] How does AIXI perform when the sets of observations $\mathcal{O}$ and actions $\mathcal{A}$ are countably infinite? Is it still self-optimizing?
10. [C35] How does AIXI perform on the sequence-prediction problem? When the environments consist purely of sequence-prediction environments, how well does AIXI do?
11. [C10] Show that replacing the posterior $w(\nu|h_{<t})$ with the prior w_ν in Theorem 7.4.2 leaves the optimal action unaffected. Show that the “constant” $\xi(e_{<t}||a_{<t})$ (7.4.6) can indeed be dropped from Theorem 7.4.2.
12. [C20] Show that there exists a universal chronological Turing machine (Definition 7.4.4).
13. [C20] Show that $\nu(e_{1:m}||a_{1:m})$ (7.4.8), $\xi(e_{1:m}||a_{1:m})$ (7.4.7), and $M(e_{1:m}||a_{1:m})$ (Theorem 7.4.5) are all chronological semimeasures.

Exercises III

14. [C20] Show that there exists a universal chronological semimeasure (Definition 6.3.1) in the sense of dominating all other semicomputable chronological semimeasures, analogous to Proposition 3.1.4.
15. [C30] Show that $M(e_{1:m} \| a_{1:m}) \stackrel{\times}{=} \xi_U(e_{1:m} \| a_{1:m})$ (7.4.9).
16. [C20] Show that $M(e_{1:m} \| a_{1:m})$ is a universal chronological semimeasure, analogous to Proposition 3.1.4.
17. [C40o] Derive a model-free version of AIXI, in the same way Q-learning is a model-free RL algorithm for MDPs.
18. [C33] AIXI has uncertainty over its future and models this with M . What if AIXI was also uncertain about its *past*? How would AIXI resolve this uncertainty, and what would this AIXI look like?
19. [C25] How well does AIXI perform on environments not in $\mathcal{M}$? *Hint*: make an assumption about how close the environment is to an environment in $\mathcal{M}$, possibly similar to Section 3.4.

Difficulty tags (e.g. [C15], [C30u]) are the book's own rating of roughly how hard/technical each exercise is, higher being harder; the trailing letters flag exercise *type* (e.g. "i" implementation-flavoured, "u" unsolved/open, "o" open research question).

History and References I

Universal artificial intelligence. The field began with [Hut00], which defined the agent AIXI, argued why AIXI is the most intelligent agent possible, showed how AIXI subsumes several interesting environment classes, provided a computable approximation of AIXI, and introduced an intelligence order relation leading to the universal intelligence measure Υ (Definition 16.7.1, later in the book). This work was extended into the tech report [Hut03d] and summarized in [Hut01e, Hut01f, Hut13b], then extended further into the book [Hut05b], which contains everything covered in this chapter. Further study into what intelligence *is* and how it should be defined, measured, and tested appears in [LH05, LH06, LH07a, LH07c, LH07b]. Universal AI as a top-down approach to building artificial general intelligence is described in [Hut07f]; many of UAI's open problems are collected in [Hut09g], some since solved. An axiomatic approach [SH11a, SH11b] demonstrates that rational agent behaviour naturally leads to AIXI. [Hut12b] surveys advances since UAI's inception and future directions; [EH18b] is a succinct, up-to-date survey of the field. Figure 7.1's taxonomy, and the proofs behind it, appear in [Hut05b, LH04]; [RH06, RH08a] develop more general criteria for (upper) self-optimizingness, based on *recoverable*, *strongly explorable*, and (worst-case) *value-stable* environments.

History and References II

Bayesian reinforcement learning. UAI is a form of *history-based* universal Bayesian reinforcement learning, though most of the wider Bayesian RL literature is studied within the narrower MDP framework; see [GMPT15] for a survey. [BVB13] shows how Bayesian inference can reduce the cost of learning an environment by factorizing the observation space, tested empirically on Atari 2600 games [BNVB13]. [SHL97] finds an optimal policy for the maximum-a-posteriori (MAP) estimate of the true environment; [Str00a] takes a similar approach, solving for the environment with highest likelihood. [RK17] takes a Bayesian approach to bandits, a special case of MDPs; [OB10b] allows the agent to model external interventions to its own behaviour.

Partial observability. Most reinforcement learning is formulated within the MDP framework, but real agents often receive only limited information about the true state (**POMDPs** — Partially Observable MDPs). [PVHR06] gives a Bayesian model-based POMDP solution and an algorithm, BEETLE, later extended to MDPs [PV08]. [RCdP07] approximates the space of all belief distributions over states (the *belief space*) via a finite-dimensional subspace to obtain ϵ -optimal solutions; both [RCdP07, PV08] factor beliefs using mixtures of Dirichlet distributions. [WLBS05] uses “sparse sampling” to approximate Bayes-optimal decision making. POTMMCP [SKH23] is an online MCTS-based planner for large POMDPs with good theoretical guarantees and practical performance in multi-agent settings. A

History and References III

heuristic market-based RL algorithm [Bau99, Bau04] — where reward is a conserved, scarce currency agents pay and receive for services — has been evaluated in POMDP environments in [KHS01b].

Causality. This book does not explicitly consider causality, which may be surprising. The actions and percepts in a history $h_{1:\infty} = a_1 e_1 a_2 e_2 \dots$ have a clear temporal order, and the chronology condition (Definition 6.3.1) already ensures actions and percepts only ever depend on the past, never the future — which is all the causal structure this book needs. There are of course deeper research questions requiring a more explicit treatment of causality [Pea00, PGJ16, PM18, PJS17] within Universal AI [OKD+21, EHKK21, CVH21, EKH19, EH18a, ELH15] and beyond [HH22]; for instance, [MWV+21] studies the difficult *credit assignment* problem of determining an action’s influence on future rewards from a counterfactual point of view.

Summary: Chapter 7

- **AI μ** (Sec. 7.1) is the fully-informed optimal agent: given the true environment μ , its action at every step is the arg max/expectation **expectimax** tree of Proposition 7.1.2, truncated to a finite horizon and approximated in practice via pruning, learned heuristics, or Monte Carlo Tree Search.
- The **Bayes mixture** $\xi = \sum_{\nu} w_{\nu} \nu$ (Def. 7.2.1) combines a whole class $\mathcal{M}$ of candidate environments into one known distribution, whose posterior weights $w(\nu|h_{<t})$ update via Bayes' rule (Def. 7.2.2) exactly as in Chapter 3, now conditioned on actions too.
- **Merging of opinions** (Thm. 7.2.6) and **on-policy value convergence** (Thm. 7.3.1) show $\xi \rightarrow \mu$ and $V_{\xi} \rightarrow V_{\mu}$ in a strong sense, with no i.i.d. or stationarity assumptions.
- **AI ξ** (Def. 7.3.2) plans against its current belief ξ instead of the unknown μ ; it is **self-optimizing** (Thm. 7.3.6) for any class $\mathcal{M}$ that admits *some* self-optimizing policy — but self-optimizing fails in general due to irrecoverable **traps** (Figure 7.1).
- **AIXI** (Def. 7.4.1) specializes $\mathcal{M}$ to every computable environment and w_{ν} to the **universal prior** $2^{-K(\nu)}$, giving the most environment-independent Bayesian agent conceivable, with an equivalent **Solomonoff/Turing-machine** formulation (Thm. 7.4.5) and a guarantee that it is never fooled into pessimism in genuinely good environments (Thm. 7.4.10).
- Chapter 8 next develops weaker, on-average optimality notions that work even where self-optimizing policies do not exist.